

Enabling natural interactions with virtual humans in XR via real-time human action recognition

Hannes Fassold*

JOANNEUM RESEARCH, DIGITAL - Institute for Digital Technologies

ABSTRACT

As Extended Reality (XR) applications evolve from passive viewing experiences to interactive simulations, the behavior of virtual humans becomes a critical factor for immersion. Users expect them to react naturally to their movements, yet implementing robust action recognition and its integration into a VR application remains a challenging task. This paper introduces a modular toolkit designed to bridge this gap. By decoupling the rendering loop from the inference process, our system allows for a sophisticated deep learning-based detection of user actions (such as hand waving or crossing hands) without compromising the frame rate of the XR experience. We present the system architecture and its integration into the Unity engine, and describe the demo applications we developed.

Index Terms: Human-Computer Interaction; Virtual Reality; Extended Reality; Artificial Intelligence; Real-time Computing; Human Action Recognition;

1 INTRODUCTION

Real-time action recognition for virtual humans in XR/AR environments is essential for creating immersive and responsive experiences and significantly enhances presence and realism. It enables intuitive human-computer interaction, allowing users to control avatars through body movements, communicate naturally with virtual characters, and receive adaptive system responses without relying on external controllers. The applications of real-time action recognition in XR/AR span numerous domains, including healthcare, gaming, education, and remote collaboration. In medical and industrial training, users can practice procedures or tasks with virtual entities that react dynamically to their actions, improving learning outcomes. In gaming and entertainment, realistic gesture-driven avatars increase immersion and engagement. For remote collaboration, recognizing non-verbal cues enables more lifelike and effective communication. Overall, real-time action recognition is a key technology for enabling engaging interactions in XR environments.

Note that there are three different types of characters that interact within XR environments, to which we collectively refer as *virtual humans*. Firstly, *smart avatars* correspond to users wearing VR headsets and serve as their digital embodiment in the virtual world. These avatars replicate the movements and gestures of the users, enabling natural interaction with both the environment and other participants. *Holoported humans* are real individuals captured using volumetric video and streamed into the XR scene, allowing for realistic telepresence. Holoportation is particularly well suited for remote collaboration, training, and social interactions. Finally, *intelligent agents* are AI-driven virtual humans that are designed to interact autonomously with users and other virtual entities.

Our proposed action recognition toolkit is the *first toolkit which is able to handle all three types of virtual humans and detect their actions in real-time* with a latency of roughly two seconds, without impacting the render rate of the VR headset negatively.

*e-mail: hannes.fassold@joanneum.at

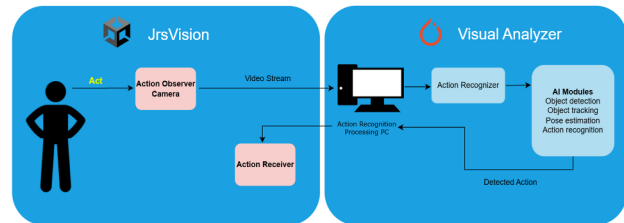


Figure 1: Overview of the proposed action recognition toolkit and its two main modules *JrsVision* and *Visual Analyzer*. The *JrsVision* Unity package renders a live video stream from the viewpoint of the observer camera and receives the detected actions. The *Visual Analyzer* does the action recognition using a combination of AI modules and returns the recognized actions.

2 ACTION RECOGNITION TOOLKIT

The proposed toolkit for real-time action recognition combines two modules: A *Visual Analyzer* Python application that receives a live video stream and does the actual real-time action recognition and sends the results to Unity, and a *JrsVision* Unity package that renders a video live stream from a certain virtual camera viewpoint (which will be processed by the *Visual Analyzer*), receives the result of the action classification via REST API or Websocket and associates them with the virtual humans in the scene. In the following, we provide more information on the two components. An illustration of the workflow can be seen in Figure 1, and a detailed toolkit description is given in [1].

Visual Analyzer: The *Visual Analyzer* Python application processes the input video stream from Unity in multiple steps. First, all virtual humans are detected using a deep learning-based object detector and tracked with an optical flow method. In parallel, their 2D poses (skeletons) are estimated using a deep-learning-based pose estimation algorithm. Actions are then recognized using deep learning-based algorithms from the 2D pose trajectories over an analysis period of around two seconds. All detected actions are then sent back to Unity. The first algorithm step detects and tracks all virtual humans visible in each video frame using the *Scaled-YoloV4* object detector combined with *TV-L1* optical flow. Next, for each detected human the 2D pose (skeleton) is calculated using the *RTM-Pose* algorithm. For skeleton-based action recognition we employ the deep-learning based *PoseConv3D* algorithm to recognize the 50 single-person actions from the NTU-60 dataset, comprising actions like hand waving, clapping, jumping, shaking head and more. Experiments demonstrate that the algorithm can robustly detect the actions of virtual humans in real time with a latency of approximately two seconds. Figure 2 illustrates successfully detected actions for different kinds of virtual humans.

JrsVision: The *JrsVision* Unity package is responsible for creating a video livestream from a dedicated Unity camera (we will call it observer camera as it observes the actions of one or several virtual humans in the Unity scene) and receiving the detected actions from the *Visual Analyzer* application and matching them with the virtual

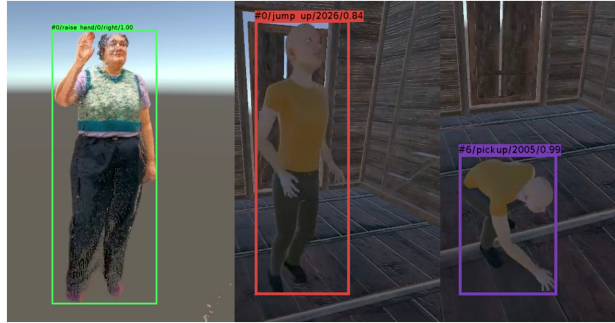


Figure 2: Illustration of successfully detected actions for different kinds of virtual humans (left one is a holoported person, whereas the middle and right one are intelligent agents).

humans in the Unity scene. In order to provide all this functionality, we developed several Unity components: The *Stream Camera Capture* component generates a low-latency video stream from the view of the Unity camera to which it is attached to. This live video stream is analysed in real-time by the Visual Analyzer python application. The *Collider Location History* component tracks the bounding box position (over the last few seconds) of the colliders which have been attached to Unity game objects of a certain tag. This is necessary for the subsequent matching step. The *Listener Api* component implements a Service which handles the REST or Websocket calls generated by the Visual Analyzer Python application. The *Action Receiver* component parses the received messages (which contain the detected actions for the virtual humans) and matches them with the Virtual Human game objects in the Unity scene which are then notified via Unity events.

3 INTERACTIVE DEMONSTRATION

We have developed two interactive VR demo applications running on a gaming notebook in PCVR mode, with a Meta Quest 3 headset connected to it via USB cable. We employ Unity version 2022.3.22 as well as Python 3.11, Pytorch 2.7 and CUDA Toolkit 12.8 for the Visual Analyzer python application. The Visual Analyzer runs on the same machine as the Unity demo. For the body tracking of the smart avatar that embodies the human user in both demos, the Meta Movement SDK is employed. The demos are running on a Windows 11 notebook with an Intel i9-14900HX CPU and a NVIDIA RTX 4090 GPU. We add a component for measuring the current render rate of the VR headset to the scene, and the measurements show that the render rate stays at 90 FPS (needed for avoiding motion sickness) even with enabled action recognition.

The first demo consists of a smart avatar (the *tourist*) and an intelligent agent (the *tour guide*). Real-time action recognition is performed for the smart avatar, and the algorithm is parametrized to detect several actions such as waving the hand, crossing hands in front of the body, or shaking the head. If an action is detected, an Unity event is triggered which could be used subsequently to determine the behavior of the tour guide. A screenshot of the demo (in third-person view) can be seen in Figure 3.

The second demo is given in the setting of a sports class and consists of a smart avatar (the *instructor*) and two intelligent agents (the *trainees*). All exercises of the smart avatar (which embodies the human user via body tracking) are repeated by the intelligent agents. As in the first demo, real-time action recognition with the same set of actions is performed for the smart avatar, and the detected action is shown on a panel integrated in the demo scene. A screenshot of the demo from the instructor view is given in Figure 4.



Figure 3: Screenshot of the first demo. The *tourist* smart avatar on the left side embodies the human user wearing the VR headset (seen in topright view), the actions (e.g. hand waving) of the tourist are detected in realtime.

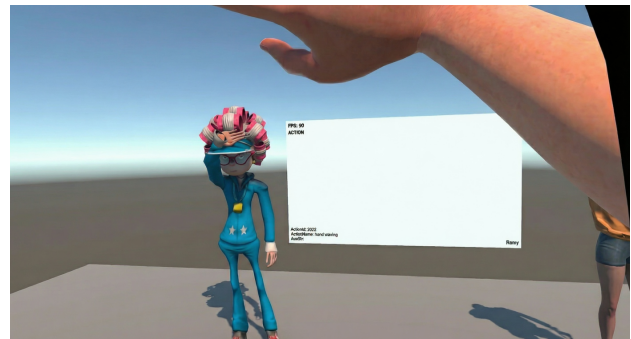


Figure 4: Screenshot of the second demo from the viewpoint of the *instructor* smart avatar. The two *trainees* are repeating the exercises performed by the instructor, and the actions of the instructor (hand waving in the screenshot) are detected in realtime and shown on the panel.

4 CONCLUSION

We address the challenges of integrating real-time action recognition into VR systems via a modular toolkit that separates rendering from inference. This design enables deep learning-based action recognition without negatively affecting XR performance. We outlined the system architecture, described its integration into the Unity engine, and showcased several demo applications. These results show that robust, real-time action recognition can be incorporated into XR environments in a practical and scalable way, supporting more responsive and believable interactions with virtual humans. In the future, we will add zero-shot action recognition with vision-language models, which will allow us to recognize novel user-defined actions without the need for training on an annotated dataset.

ACKNOWLEDGMENTS

We would like to thank Amir Dini for his expertise and input in developing the demo applications. Thanks goes also to Adobe and DIDIMO for the human 3D characters used in the demos. This work has received funding from the European Union's Horizon Europe research and innovation programme under grant No. 101135025, "Presence XR".

REFERENCES

- [1] H. Fassold. A flexible toolkit for real-time action recognition of virtual humans in XR/AR environments. In *EAIM*, June 2025.